ASSIGNMENT 5 – CO5

PART A | TASK A1

DESIGN A FILE FORMAT

Transport Office Bus-Pass Records

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

Design report containing record layouts, operation pseudocode, damaged-data handling, and an alternative fixed-length RandomAccessFile analysis.

1. Problem Understanding

The transport office stores approximately 8,000 bus-pass records each year. Each logical record contains seven fields: pass ID, student register number, name, route number, boarding stop, validity date and amount paid. The file-based design must support adding a pass, finding a pass by pass ID, and listing every pass on a specified route.

Design decision

Use a UTF-8 delimited text file for the primary design. Store one pass per line in CSV-style format, use a buffered character stream for line-oriented processing, and validate each record before using it. A text file is human-readable and easy to inspect. Sequential search is acceptable for this assignment's scale, although it becomes slower as the file grows.

2. A1.1 – Represent the Record

A. Delimited text-file layout

Use UTF-8 encoding, one record per line, and a comma (,) as the delimiter. The seven fields must appear in this exact order:

Position	Field	Representation / validation
1	passId	Non-empty text; unique identifier
2	registerNumber	Non-empty text; preserve leading zeros
3	name	Text; may contain commas or quotation marks
4	routeNumber	Integer
5	boardingStop	Text; may contain commas or quotation marks
6	validityDate	ISO date: YYYY-MM-DD
7	amountPaid	Decimal numeric value; use a consistent decimal point

Example record: `BP2045,192525148,"Ananya, R.",12,"Lake View, Gate 2",2027-03-31,4500.00`

Handling a name containing the delimiter

Apply CSV quoting rules. Enclose a field containing a comma, quote or line-break in double quotation marks. Inside a quoted field, represent each literal double quote as two double quotes. For example, the name Ananya, R. is stored as `"Ananya, R."`. Do not split a line on every comma blindly; use a CSV-aware parser that understands quoted fields. The same rule applies to boardingStop.

B. Binary-file layout using DataOutputStream

Write fields in the same logical order. DataOutputStream writes primitive values in a portable binary representation and writeUTF() stores a length-prefixed modified UTF-8 string.

Order	Field / method	Stored size
1	writeUTF(passId)	Variable: 2-byte length prefix + encoded bytes
2	writeUTF(registerNumber)	Variable: 2-byte length prefix + encoded bytes
3	writeUTF(name)	Variable: 2-byte length prefix + encoded bytes
4	writeInt(routeNumber)	4 bytes
5	writeUTF(boardingStop)	Variable: 2-byte length prefix + encoded bytes

Order	Field / method	Stored size
6	writeUTF(validityDate)	Variable: 2-byte length prefix + encoded bytes; YYYY-MM-DD
7	writeDouble(amountPaid)	8 bytes (IEEE 754 double)

The binary record is variable-length because writeUTF() stores strings of varying lengths. The writer and reader must use exactly the same field order and matching methods (readUTF, readInt, readUTF, readUTF, readUTF, readDouble). DataOutputStream does not automatically add record separators; the reader must consume fields sequentially. writeUTF() has a maximum encoded payload length of 65,535 bytes per string.

3. A1.2 – Design the Operations (Text-File Design)

Stream selection and justification

FileReader / FileWriter: character streams suitable for text files. Specify UTF-8 explicitly through an appropriate reader/writer construction so that encoding is consistent. **BufferedReader / BufferedWriter:** wrap the character streams to reduce small read/write operations and provide readLine() and efficient writing. **FileInputStream / FileOutputStream:** byte streams are not the preferred choice for this text-file design; use them when raw bytes are required, such as copying a file or processing a binary format.

Pseudocode conventions

The following is algorithmic pseudocode, not Java source code. A CSV-aware parse routine is assumed; it returns fields while respecting quoted delimiters and doubled quotation marks.

```
ALGORITHM ADD_PASS(pass)
    VALIDATE all seven fields in pass
    IF any field is invalid THEN
        REPORT "Invalid pass record"
        RETURN
    ENDIF

    OPEN passFile using FileWriter in APPEND mode
    WRAP FileWriter with BufferedWriter
    FORMAT pass as one CSV record in the specified field order
    WRITE formatted record
    WRITE line separator
    FLUSH BufferedWriter
    CLOSE BufferedWriter
    REPORT "Pass added"
END ALGORITHM

ALGORITHM FIND_PASS_BY_ID(targetPassId)
    IF passFile does not exist THEN
        REPORT "Pass file not found"
        RETURN NOT_FOUND
    ENDIF

    OPEN passFile using FileReader
    WRAP FileReader with BufferedReader

    WHILE a line can be read
        IF line is empty THEN
            CONTINUE
        ENDIF

        fields <- CSV_PARSE(line)
        IF parsing fails OR number of fields is not 7 THEN
            RECORD line number as damaged
            CONTINUE
        ENDIF

        IF required fields are empty OR route/date/amount is invalid THEN
            RECORD line number as damaged
            CONTINUE
        ENDIF

        IF fields[1] equals targetPassId THEN
            CLOSE BufferedReader
            RETURN fields
        ENDIF
    ENDWHILE

    CLOSE BufferedReader
    RETURN NOT_FOUND
END ALGORITHM

ALGORITHM LIST_PASSES_BY_ROUTE(targetRoute)
    IF passFile does not exist THEN
        REPORT "Pass file not found"
        RETURN empty list
    ENDIF
```

```
    matchingPasses <- empty list
    OPEN passFile using FileReader
    WRAP FileReader with BufferedReader

    WHILE a line can be read
        IF line is empty THEN
            CONTINUE
        ENDIF

        fields <- CSV_PARSE(line)
        IF parsing fails OR number of fields is not 7 THEN
            RECORD line number as damaged
            CONTINUE
        ENDIF

        IF required fields or numeric/date values are invalid THEN
            RECORD line number as damaged
            CONTINUE
        ENDIF

        IF integer value of fields[4] equals targetRoute THEN
            ADD fields to matchingPasses
        ENDIF
    ENDWHILE

    CLOSE BufferedReader
    RETURN matchingPasses
END ALGORITHM
```

Complexity of the three text-file operations

Let n be the number of records and L be the average encoded line length. Adding a record in append mode is O(L) for formatting and writing (amortized O(1) record append, excluding storage-device effects). Finding by ID is O(n) in the worst case because records are scanned sequentially. Listing a route is O(n) because every record must be checked. The scan operations use O(L) auxiliary memory when processing one line at a time; the route listing additionally uses O(k × L) memory to hold k matching records, or can stream results to keep auxiliary memory at O(L).

4. A1.3 – Design for Damaged Data

Damaged line: Parse each line independently. Treat a CSV parsing error, a field count other than seven, an empty required field, an invalid integer route, an invalid ISO date, or a non-numeric/negative amount as a damaged record. Record its line number and reason in a separate error log, skip it, and continue with the next line. Never allow one malformed line to terminate the entire search or route listing.

Missing file: Before a read operation, check whether the file exists. If it does not, report a clear “file not found” condition. For a listing operation, return an empty result only if the application explicitly treats a not-yet-created store as empty; otherwise report the missing-file condition. The add operation can create the file when opening the writer in create/append mode.

Incomplete write / power failure: Write one complete record followed by a line terminator, flush the buffered writer, and close it promptly. On recovery, detect an unterminated final line and treat it as incomplete; validate it and move it to a recovery/error file rather than accepting it as a complete pass. For stronger protection against corruption that still looks syntactically valid, add a checksum per record or maintain a write-ahead/recovery log. A flush alone cannot guarantee that data has reached durable storage; critical systems should use an appropriate force-to-disk strategy and backups.

5. Alternative Concept – RandomAccessFile with Fixed-Length Records

With fixed-length records, every field is allocated a predetermined number of bytes. Text fields are encoded to a fixed width and padded (for example, with spaces); values longer than the allotted width must be rejected or handled by an explicit overflow rule. Numeric fields can use fixed-width binary values. If the record size is R bytes

and records are numbered from zero, record i begins at byte offset i × R. RandomAccessFile.seek(i × R) positions the file pointer directly at that record.

Finding by pass ID still requires a way to know the record index. Without a separate index, the program must scan records and compare IDs, so finding remains O(n); seeking to a known record index is O(1). If a sorted index or a hash index maps pass ID to record number, lookup can be faster, but that index must also be maintained consistently when records are added or changed. Listing all passes on a route remains O(n) without a route index.

Operation / property	Delimited text file	Fixed-length RandomAccessFile
Find by ID (no index)	O(n) scan	O(n) scan; O(1) seek only after record index is known
Add	Append one line; O(L) write	Seek to end and write R bytes; O(R)
List by route	O(n) scan	O(n) scan
Space use	Variable-length; generally compact	Exactly R bytes per record; padding wastes unused bytes
Human readability	High; easy to inspect/edit	Low; binary and format-dependent

Padding overhead

If a name is allocated 40 bytes but uses only 12 bytes, 28 bytes are unused for that record. The same waste can occur in register number, boarding stop and other fixed-width fields. If fields are allocated too narrowly, valid data may not fit; if allocated too generously, the file wastes storage. For 8,000 records, even a small amount of padding per record accumulates across the file.

6. Conclusion

A UTF-8 CSV-style text file with buffered character I/O is a straightforward design for the transport office's stated file operations. Quoted-field parsing protects names and boarding stops that contain commas. Validation and recovery handling prevent a damaged record from stopping the entire process. Sequential ID lookup and route listing are O(n); a fixed-length RandomAccessFile provides O(1) positioning only when the record index is already known, while padding trades storage efficiency for predictable record size and direct seeking.

Submission note

Task A1 is a design-and-pseudocode task. The assignment explicitly requires algorithmic pseudocode for Part A and states that Java code will not be accepted in place of pseudocode. Therefore, no Java implementation is required for this task.